

Двоична пирамида е попълнено двоично дърво, в което върси едно от убоите:

- Ключът на всеки връх е по-голям или равен на ключа на дъщерите му (в този случай имаме максимална пирамида)
- Ключът на всеки връх е по-малък или равен на ключа на дъщерите му (в този случай имаме минимална пирамида)

### Свойства

- височината на пирамидата е  $\lfloor \log_2 n \rfloor$
- височината на  $A[i]$  е на четирици  $\lfloor \frac{i}{2} \rfloor$
- височината на  $A[i]$  е на четирици  $2i$
- височината на  $A[i]$  е на четирици  $2i+1$

### Naive Binary Heap ( $A[1..n]$ )

- ```
01 for i ← 2 to n
02   k ← i
03   while A[parent(k)] > A[k]
04     swap(A[parent(k)], A[k])
05     k ← parent(k)
```

Ще въведем дефиниция почти-пирамида

Почти-пирамида е попълнено дърво, чиито ключове удовлетворяват пирамидалното свойство с възможно изключение на ний-дъщерното листо на последния ред

$I(i) \Leftrightarrow A[1..i]$  е пирамида

• База ( $i=2$ )

$A[1..2]$  - пирамида е ✓

• Поддръжка

Допускаме че  $A[1..i-1]$  е пирамида. Добавянето на нов елемент на позиция  $i$  създава почти-пирамида, в която единствените възможни

инверсии са по пътя от новия елемент до корена и while цикълът пренахва тези инверсии (показано по-долу). След while цикъла  $A_{i+1}[1..i+1] = A_i[1..i]$  е пирамида

#### • Терминация

Цикълът приключва при  $i=n+1 \Rightarrow I(i) = A[1..i-1] = A[1..n+1-1] = A[1..n]$  е пирамида

$J(m) \Leftrightarrow A_m[1..i]$  е почти-пирамида, като единственото възможно решение е лесу излюбващия връх на позиция  $k_m$  и родителя му на  $\lfloor \frac{k_m}{2} \rfloor$

#### • База ( $m=1$ )

$k_1=1$ . От инварианта на външния цикъл знаем, че  $A_1[1..i-1]$  е пирамида.

След добавянето на  $A_1[i]$  към пирамидата, той е листо  $\Rightarrow$  няма деца с които да е в конфликт  $\Rightarrow$  може да е в конфликт само с родителя си

#### • Поддръжка

Допускаме, че инвариантът е в сила за стъпка  $m$ . Поново взимаме в цикъла  $\Rightarrow A_m[\text{parent}(k_m)] \leq A[k_m]$ . Ни рече че ги разменяме. Поново  $A_m[k_m] \geq A_m[\text{parent}(k_m)]$  и поново от външната инварианта  $A[1..i-1]$  е пирамида, то  $A_m[\text{parent}(k_m)]$  е дъщеря от другото си дете и от транзитивност на  $\geq$  не създаваме нови конфликти. Така получихме че с това размястване не правим нови конфликти защото и двете деца са  $\leq$  от елемента който придобихме чичоре. Ни рече че  $k_{j+1} \leftarrow \text{parent}(k_j)$

#### • Терминация

Цикълът приключва когато стигне корен (няма родител с който да е в конфликт) или  $A[\text{parent}(k)] \geq A[k]$ , което обаче значи че няма повече конфликти и  $A[1..i]$  - пирамида

Външния цикъл прави  $n-2+1=n-1$  стъпки, а вътрешния в най-лошия случай ще изкара листо так до корена. Поново височината е  $\log_2 n$ .

$$\text{Получаваме } (n-1)(\lfloor \log_2 n \rfloor) = n \cdot \lfloor \log_2 n \rfloor - \lfloor \log_2 n \rfloor = n \cdot \lfloor \log_2 n \rfloor - \lfloor \log_2 n \rfloor \in O(n \cdot \log n)$$

BuildHeap( $A[1..n]$ )

e1 for  $i \leftarrow \lfloor n/2 \rfloor$  downto 1

e2     Heapify( $A, i$ )

Heapify( $A[1..n], i$ )

e1      $j \leftarrow i$

e2     while  $j \leq \lfloor n/2 \rfloor$

e3          $\text{left} \leftarrow \text{Left}(j)$

e4          $\text{right} \leftarrow \text{Right}(j)$

e5          $\text{largest} \leftarrow j$

e6         if  $A[\text{left}] > A[\text{largest}]$

e7              $\text{largest} \leftarrow \text{left}$

e8         if  $\text{right} \leq n$  &&  $A[\text{right}] > A[\text{largest}]$

e9              $\text{largest} \leftarrow \text{right}$

e10         if  $\text{largest} \neq j$

e11             swap( $A[\text{largest}], A[j]$ )

e12              $j \leftarrow \text{largest}$

e13         else

e14             break

Инвариант на BuildHeap

При всяко достигане на ред  $i$ ,  $A[i+1], A[i+2], \dots, A[n]$  са пирамиди

- База ( $i = \lfloor n/2 \rfloor$ )  $A[\lfloor n/2 \rfloor + 1], A[\lfloor n/2 \rfloor + 2], \dots, A[n]$  - пирамиди - да, защото това са листата

- Погребълка

Допускаме, че инвариантът е верен за някое  $i \Rightarrow \text{Left}(i)$  и  $\text{Right}(i)$

са пирамиди и можем да приложим Heapify. След Heapify получаваме  $A[i]$  е пирамиди. Така след целочисленитранс на  $i$  инвариантът е верен

• Терминация

Уиклазт прикатозва при  $i=0$  и според инварианта  $A[1], A[2], \dots, A[n]$  са пирамиди, но  $A[i]$  е корена  $\Rightarrow$  имаме пирамида

Инвариант на Heapify

При всяко достигане на реч  $A[L(j)]$  и  $A[R(j)]$  са пирамиди (ако същ.)

Всички пирамидални инверсии в  $A[i]$  се кичират в  $A[j]$

Не е много изчерпателно  $\downarrow \downarrow \downarrow$

• База ( $i=j$ ) Поневже BuildHeap започва от родителите на листата, то всички лявото и дясното поддърво са пирамиди

• Процедура

Служия в които се показва те каквито и инверсии да има между  $j$  и дясната му инварианта се започва

• Терминация

Стигаме до листо  $j > \lfloor n/2 \rfloor$ , а листата нямат дъщеря

Стигаме момент в който  $j = \text{largest}$  (т.е. елементът си е кичерил лявото) и така понеже според инварианта инверсиите си само в текущия връх а в него няма  $\Rightarrow$  пирамида

Половината върхове са листа (но тях не ги разглеждаме)

$\frac{1}{4}$  са родители на листата

$\frac{1}{8}$  са родители на родители на листата

...

само един е корена

Имаме  $n \cdot \left( 0 \cdot \frac{1}{2} + \frac{1}{4} \cdot 2 \cdot \frac{1}{2} + 3 \cdot \frac{1}{4} + \dots + h \cdot \frac{1}{2^{h+1}} \right) = n \cdot 2 \in O(n)$

$\underbrace{\hspace{10em}}$   
 $2$   
  
ДДС неща

HeapSort( $A[1..n]$ )

01 BuildHeap( $A$ )

02 for  $i \leftarrow n$  downto 2

03     swap( $A[i], A[1]$ )

04     Heapify( $A[1..i-1], 1$ )

Инвариант преди всяко достигане на 02

1.  $A[i+1..n]$  съдържа  $n-i$ -те най-големи елемента на оригиналния масив във възходящ ред (сортирани)

2.  $A[1..i]$  е пирамида

• База ( $i=n$ )

$A[n+1..n]$  - празен  $\Rightarrow$  сортиран  $\checkmark$

$A[1..n]$  - пирамида  $\checkmark$

• Поуредовка

Допускаме инварианта за  $i$ .

От 2.  $\Rightarrow A[i]$  е най-големият елемент

От 1.  $\Rightarrow A[i+1..n]$  са  $n-i$ -те най-големи елемента сортирани във възходящ ред.  $\Rightarrow A[i+1] \leq A[i+2] \leq \dots \leq A[n]$

Понеже  $A[i]$  е най-голям в  $A[1..i]$  и  $A[i+1..n]$  са най-големите  $n-i$  в  $A[1..n]$  сортирани във възходящ ред, в частност  $A[i+1] \geq A[i]$ , а на ред 03

$A[i]$  и  $A[i+1]$  се сменят, и понеже  $A[i] \leq A[i+1] \leq A[i+2] \leq \dots \leq A[n] \Rightarrow$  1. се запазва.

На ред 04 Heapify превръща  $A[1..i-1]$  в пирамида  $\Rightarrow$  2. се запазва

Терминация ( $i=1$ )

$A[2..n]$  съдържа  $n-1$  най-големи елемента сортирани във възходящ ред  $\Rightarrow$

$A[1]$  - минимум.  $\Rightarrow A[1..n]$  е сортиран

BuildHeap е  $O(n)$

Heapify е  $O(\log n)$

for цикълът прави  $n-1$  итерации  $\Rightarrow O(n) + (n-1) \cdot O(\log n) = O(n) + O(n \log n) = O(n \log n)$

HeapSort e in-place, но не е стабилен

HeapSort( $A[1..n]$ )

BuildHeap( $A[1..n]$ )

for  $i \leftarrow n$  downto 2

    swap( $A[i]$ ,  $A[1]$ )

    Heapify( $A[1..i-1]$ , 1)